

CoppeliaSim Tutorial 5

外力 (Force) を加える：押して倒す

Tutorial 4ではモーターで関節を「内部から」動かしましたが、今回は**外部から力を加えて**物体を動かす方法を学びます。これがGoalで目指す「外力負荷シミュレーション」の基礎になります。

目的

- **外力 (External Force)** をオブジェクトに適用する方法を理解する
 - **Luaスクリプト** を使って力を動的に制御する
 - 力の **大きさ・方向・作用点** を変えて挙動の違いを観察する
-

使用環境

- CoppeliaSim (Bullet 物理エンジン)
 - **前提:** Tutorial 1~4 を完了していること
-

1. シーンの準備

今回は新しいシンプルなシーンを作ります（振り子より分かりやすいため）。

1.1 新規シーンを作成

1. **File → New scene**

1.2 床の確認

新規シーンにはデフォルトで **Floor** が配置されています。そのまま使用します。

 **確認:** Scene hierarchy に **Floor** があることを確認してください。

1.3 ターゲット（押される物体）を追加

1. **Add → Primitive shape → Cuboid**
2. サイズ: **X=0.1, Y=0.1, Z=0.3**（縦長の直方体）
3. ☒ **Create dynamic and responsible shape** にチェック
4. 位置を調整: **X=0, Y=0, Z=0.15**（床の中央に立つように）
5. Scene hierarchy で名前を **Target** に変更
6. **Dynamic properties** ダイアログを開き、**Mass** (質量) を **0.2** に設定

 **重要:** デフォルトの3.0kgでは重すぎて5Nの力では倒れません。0.2kg (200g) 程度に軽くしましょう。

1.4 シミュレーションテスト

1. ツールバーの  **Toggle real-time mode** (ウサギのアイコン) をクリックしてONにする（実時間再生になり、動きが見やすくなります）
2. ▶ **Start simulation**
3. **Target** が床の上に安定して立っていればOK
4.  **Stop simulation**

2. 外力を加えるスクリプトを作成

2.1 スクリプトを追加

1. Scene hierarchy で **Target** を右クリック
2. **Add** → **Script** → **Simulation script** → **Non-threaded** → **Lua**

💡 **Non-threaded vs Threaded:** Non-threaded はシミュレーションのメインループと同期して実行されます。今回のような単純な力の適用には Non-threaded が適しています。

2.2 スクリプトを編集

1. 追加されたスクリプトをダブルクリックして開く
2. 以下のコードに **置き換え** る：

```
function sysCall_init()
    -- Get parent object handle
    targetHandle = sim.getObject('..')

    -- Delay before applying force (seconds)
    forceDelay = 1.0

    -- Flag to check if force has been applied
    forceApplied = false

    print("=== Force Demo Ready ===")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    -- Apply force after delay (only once)
    if t >= forceDelay and not forceApplied then
        -- Force vector [X, Y, Z] (Newton)
        local force = {5, 0, 0} -- Apply 5N force in X direction

        -- Point of action (relative to object center)
        local position = {0, 0, 0.1} -- Apply force at the top

        -- Apply force
        sim.addForce(targetHandle, position, force)

        forceApplied = true
        print(string.format("Force applied at t=%.2f sec: [%.1f, %.1f,
```

```
%.1f] N",
    t, force[1], force[2], force[3]))
end
end
```

2.3 コードの解説

関数/変数	説明
<code>sim.getObject('..')</code>	親オブジェクト (Target) を取得
<code>sim.getSimulationTime()</code>	シミュレーション開始からの経過時間 (秒)
<code>sim.addForce(handle, pos, force)</code>	指定オブジェクトに力を加える
<code>force = {5, 0, 0}</code>	X方向に5ニュートンの力
<code>position = {0, 0, 0.1}</code>	オブジェクト中心より0.1m上に作用点

3. シミュレーション実行

- 1. ▶ **Start simulation**
- 2. 1秒後に **Target** が **パタン!** と倒れるはずです
- 3. コンソールに **Force applied at t=1.00 sec: [5.0, 0.0, 0.0] N** と表示されます

💡 **観察ポイント:** 上部 (Z=0.1) に力を加えているので、回転しながら倒れます。これが「モーメント (回転力)」の効果です。

4. 実験：パラメータを変えてみよう

実験1: 力の大きさを変える

```
local force = {1, 0, 0}    -- 弱い力 (揺れるだけ?)
local force = {10, 0, 0}   -- 強い力 (吹っ飛ぶ)
local force = {50, 0, 0}   -- もっと強い力
```

実験2: 力の方向を変える

```
local force = {0, 5, 0}    -- Y方向 (横) に押す
local force = {0, 0, 10}   -- Z方向 (上) に持ち上げる
local force = {5, 5, 0}    -- 斜め方向
```

実験3: 作用点を変える

```

local position = {0, 0, 0}      -- 重心に力を加える（倒れにくい）
local position = {0, 0, 0.15}  -- 最上部（最も倒れやすい）
local position = {0, 0, -0.1}  -- 下部（倒れにくい）

```

 **考察:** 同じ大きさの力でも、作用点が高いほど倒れやすいことを確認してください。これは理学療法で「重心より高い位置に外乱を加えると姿勢崩壊しやすい」という臨床知見と一致します。

5. 応用：継続的に力を加える

1回だけでなく、一定時間力を加え続けるパターンも試してみましょう。

前のスクリプトを消さずに残しておきたい場合は、**モード切替** できるように書き換えるのがおすすめです。以下のスクリプトは、先頭の `mode` という変数を書き換えるだけで挙動を変えられます。

```

function sysCall_init()
    targetHandle = sim.getObject('..')

    -----
    -- CONFIGURATION
    -----
    -- Select Mode: 1 = One-shot Impulse, 2 = Continuous Force
    mode = 2
    -----

    -- Parameters for Mode 1 (Impulse)
    impulseDelay = 1.0
    impulseApplied = false

    -- Parameters for Mode 2 (Continuous)
    contStartTime = 1.0
    contDuration = 0.5

    print("=== Force Demo Ready (Mode " .. mode .. ") ===")
end

function sysCall_actuation()
    local t = sim.getSimulationTime()

    -- Mode 1: One-shot Impulse
    if mode == 1 then
        if t >= impulseDelay and not impulseApplied then
            local force = {5, 0, 0}
            sim.addForce(targetHandle, {0, 0, 0.1}, force)
            impulseApplied = true
            print(string.format("Impulse applied at %.2f s", t))
        end
    end

    -- Mode 2: Continuous Force
    elseif mode == 2 then

```

```
if t >= contStartTime and t < contStartTime + contDuration then
    local force = {3, 0, 0}
    sim.addForce(targetHandle, {0, 0, 0.1}, force)
    -- No 'applied' flag needed because we want it every frame
end
end
end
```

💡 解説：2つの書き方の違い

なぜ Mode 1 と Mode 2 で書き方が違うのでしょうか？

モード	やりたいこと	制御の方法	イメージ
Mode 1 (衝撃)	一瞬だけ力を加えたい	フラグ (impulseApplied) を使う。 「まだ力を加えてない？」→「なら押す！そして『もう押した』と記録」	ボールを蹴る 🏈 (一瞬のインパクト)
Mode 2 (継続)	一定時間押し続けたい	時間の範囲 (Start ~ End) を使う。 「今は押す時間帯に入ってる？」→「なら押す（フラグは不要）」	台車を押す 🛒 (じわ〜っと押し続ける)

- **Mode 1 のポイント:** `if ... and not impulseApplied` があるため、1回実行されると `true` になり、次のフレームからは無視されます。
- **Mode 2 のポイント:** 時間 `t` が範囲内にある限り、毎フレーム（1秒間に20回くらい）`sim.addForce` が実行され続けます。

💡 **理学療法との関連:** これは「介助者がゆっくり押す」場合と「瞬間的に外乱が加わる」場合の違いをシミュレートできます。

6. シーンを保存

1. **File → Save scene as...**
2. ファイル名: `force_demo.ttt`

🔧 トラブルシューティング

症状	チェック項目
何も起きない	スクリプトが正しく追加されていますか？ <code>sysCall_actuation</code> は毎フレーム呼ばれます
エラーが出る	<code>sim.addForce</code> の引数は <code>(handle, {x,y,z}, {fx,fy,fz})</code> の順です
オブジェクトが動かない	<code>Create dynamic</code> にチェックを入れましたか？静的オブジェクトには力が効きません
倒れない	<code>Mass</code> が重すぎませんか？(例: 3.0 -> 0.2) / 力が弱すぎませんか？

症状	チェック項目
エラーが出る	<code>sim.getObject('..')</code> になっていますか？ 。だとスクリプト自身を取得してしま います
床をすり抜ける	<code>Create_respondable</code> にチェックを入れましたか？

物理的な補足

sim.addForce vs sim.addForceAndTorque

- `sim.addForce`: 特定の位置に力を加える（自動的にトルクも発生）
- `sim.addForceAndTorque`: 力とトルクを個別に指定

単位について

- 力: ニュートン (N)
- 位置: メートル (m)
- 質量: キログラム (kg) ← デフォルトでオブジェクトには質量が設定されています

△ 重要：力の向きについて（相対座標 vs 絶対座標）

今回の `sim.addForce` 関数は、**相対座標（ローカル座標）** で計算されます。

- **Force {5, 0, 0}**: 「箱にとっての 前方」に5N
- **Position {0, 0, 0.1}**: 「箱の 中心から上0.1m」の位置

つまり、箱が倒れ始めると、**力の向きも一緒に傾きます**。イメージとしては、「外から風で押されている（ワールド座標）」のではなく、「**箱に取り付けたロケットエンジン（スラスター）が噴射している**」状態に近いです。（外から一定方向の風のような力を加えたい場合は、`sim.addForceAndTorque` を使い、座標変換を行う必要があります）

次へのステップ

これで「外力を加える」基礎ができました！

次のチュートリアルでは、この技術を発展させて：

- **複数のリンクを持つモデル** に外力を加える
- **転倒を検知するセンサー** を追加する
- **力の大きさをリアルタイムに変化させる UI** を作る

などに挑戦していきます。

復習問題

1. `sim.addForce` の3つの引数は何ですか？
2. 作用点を重心より上にすると、なぜ倒れやすくなりますか？
3. 力を1回だけ加える場合と、継続的に加える場合で、コードはどう違いますか？

